

NETWORK APPLICATION PROTOCOL

DART v1

Deadline-Aware Reliable Telemetry

เลือกความน่าเชื่อถือให้เหมาะสมกับความเร่งด่วนของแต่ละข้อความ

01418351 • PROJECT 1: SOCKET PROGRAMMING

รายวิชา: หลักการสื่อสารคอมพิวเตอร์และการประมวลผลบนคลาวด์

Course: Computer Communications and Cloud Computing Principles

ชื่อ-นามสกุล: เมตัส พานิช กายย์

รหัสนิสิต: 6710405460

เอกสารฉบับนี้อธิบาย protocol, source code, การทดสอบ, benchmark และแนวทางสาธิต โดยใช้ virtual sensors จึงไม่ต้องใช้ hardware

SECTION 01

ภาพรวมงานและข้อเสนอหลัก

DART เป็น application-layer protocol ที่สร้างขึ้นเองบน UDP สำหรับ telemetry จาก sensor หลายตัวไปยัง server

แนวคิดสำคัญ: ข้อมูลทุกชิ้นไม่ควรเสียค่าใช้จ่ายด้าน **reliability** เท่ากัน

สิ่งที่ protocol ทำ

- รวมค่าปกติหลายค่าเป็น **DATA_BATCH** เพื่อลด overhead ต่อ reading
- ส่งค่าที่เปลี่ยนเร็วแบบ **LATEST_UPDATE** โดย server เก็บเฉพาะ sequence ใหม่กว่า
- ส่งเหตุฉุกเฉินแบบ **CRITICAL_ALERT** พร้อม ACK, timeout, retransmission และ duplicate suppression
- ลงทะเบียน sensor เป็น session และรองรับหลาย connection แข็งตรรกะพร้อมกันด้วย worker pool

ขอบเขต

Prototype นี้เป็นงานการศึกษา ไม่ใช่มาตรฐานอินเทอร์เน็ต และไม่ได้อ้างว่าเร็วกว่า TCP/HTTP ทุกสถานการณ์ จุดประสงค์คือแสดงให้เห็นการออกแบบ syntax, semantics, rules และ trade-off ที่วัดได้ตามเนื้อหา Chapter 1-3

สิ่งที่ส่งมอบ

ส่วน	หลักฐาน
Protocol design	เอกสารฉบับนี้และ docs/PROTOCOL_SPEC.md
Source code	client, server, simulator, benchmark และ Wireshark dissector
Demo	demo.py พร้อม PASS oracle และ results/latest_demo.json
Presentation	DART_Presentation.pptx พร้อม speaker notes และ VIDEO_SCRIPT.md

SECTION 02

ปัญหา วัตถุประสงค์ และการใช้งาน

ตัวอย่างคือระบบเฝ้าระวังโรงงานจำลอง แต่ protocol ใช้กับ telemetry ประเภทอื่นได้

ปัญหา

ระบบ sensor ส่งข้อมูลหลายลักษณะพร้อมกัน: ค่าอุณหภูมิปกติจำนวนมาก, ตำแหน่งที่ค่ากำหนดประโยชน์เร็ว และสัญญาณไฟไหม้ที่ห้ามหาย หากบังคับให้ทุกข้อความรอ ACK จะได้ความน่าเชื่อถือแต่เกิด ACK traffic, retry และ head-of-line delay ที่ไม่จำเป็น หากไม่ใช้ ACK เลย เหตุสำคัญอาจสูญหาย

วัตถุประสงค์

- ออกแบบ protocol ที่แยก policy ตามความหมายของข้อความ ไม่ใช่ตาม socket ทั้งเส้น
- แสดง request/response, status code/status phrase และ message log ที่อ่านได้
- วัด normal delivery, latest final state, critical acceptance/confirmation, latency และ bytes ต่อ generated value
- ทำให้สาธิตได้ด้วย software-only virtual sensors บนเครื่องเดียว

ตัวอย่างการใช้

ข้อมูล	ความต้องการ	DART policy
อุณหภูมิปกติ	ปริมาณมาก ยอมเสียบางส่วน	BEST_EFFORT_BATCH
ตำแหน่งล่าสุด	ค่าปัจจุบันสำคัญกว่าประวัติ	LATEST_ONLY
ไฟไหม้	ต้องยืนยันและไม่ประมวลผลซ้ำ	CRITICAL_RELIABLE
ลงทะเบียน/ตั้งค่า	ต้องมี response ชัดเจน	CONTROL

การสาธิตไม่ต้องใช้ sensor จริง: โปรแกรมสร้างค่าและเหตุการณ์ตามเวลา แล้วส่งผ่าน UDP socket จริงบน localhost

SECTION 03

เหตุผลที่เลือก UDP

DART ต้องการ datagram boundary, low baseline overhead และสิทธิ์กำหนด reliability เองใน application layer

ประเด็น	UDP	TCP	ผลต่อ DART
การเชื่อมต่อ	ไม่ต้อง handshake	connection-oriented	virtual sensor เริ่มส่งได้เร็ว
ขอบเขตข้อความ	หนึ่ง datagram ต่อหนึ่ง message	byte stream	header/payload แยก packet ได้ตรงไปตรงมา
Reliability	application เลือกเอง	reliable ordered byte stream	เลือก ACK เฉพาะ critical/control
ข้อเสีย	อาจ loss, reorder, duplicate	ควบคุมให้แล้ว	DART ต้องมี CRC, sequence, TTL, ACK, timer

ข้อสรุปการเลือก transport

DART เลือก UDP ไม่ใช่เพราะ UDP ดีกว่า TCP โดยทั่วไป แต่เพราะโจทย์ต้องการทดลอง reliability ระดับข้อความ: normal data ยอม loss, latest data ยอมทิ้งค่าเก่า และ critical data ใช้ ACK/retry เฉพาะจุด การเลือกนี้ทำให้เห็นแนวคิด Chapter 2 เรื่อง transport service requirements และ Chapter 3 เรื่อง UDP, checksum, multiplexing/demultiplexing และ reliable data transfer

ข้อจำกัดจากการเลือก UDP

- ไม่มี congestion control, flow control หรือ path MTU discovery แบบสมบูรณ์
- ขนาด DART datagram จำกัด 1,200 bytes เพื่อลดความเสี่ยง fragmentation
- Reliability ของ prototype ใช้ stop-and-wait ต่อ reliable call; งานจริงควรพิจารณา windowing และ security

DART คือ protocol บน UDP ไม่ใช่การเปรียบเทียบว่า DART เร็วกว่า UDP

SECTION 04

สถาปัตยกรรมและ logical connection

Client และ server เป็นคนละ process; IP address + UDP port ระบุ socket ปลายทางตาม Chapter 2

ลำดับ	Virtual sensor client	DART server
1	ส่ง REGISTER_REQ โดย session_id = 0	ออก session_id และตอบ REGISTER_RES 201
2	ส่ง DATA_BATCH / LATEST_UPDATE / CRITICAL_ALERT	ตรวจ header, CRC, session, TTL และ payload
3	รอ response เฉพาะ message ที่ต้อง reliable	ตอบ ACK 202/409/408 ตามผล
4	timeout แล้วส่ง sequence เดิมพร้อม RETRANSMISSION	duplicate cache ป้องกัน side effect ซ้ำ

Multi-sensor concurrency

- Simulator สร้าง SensorClient หลายตัว แต่ละตัวมี UDP socket, session และ sequence ของตนเอง
- Server receiver รับ datagram แล้วส่งงานไป ThreadPoolExecutor; state ที่ใช้ร่วมกันป้องกันด้วย lock/condition
- In-flight duplicate ของ identity เดียวกันรอผล packet แรก; cache จะ commit เฉพาะเมื่อการประมวลผลสำเร็จ
- wait_until_idle ตรวจ active task และ quiet period ก่อน snapshot เพื่อไม่ตัดผลลัพธ์เร็วเกินไป

ที่อยู่และ session

Server ผูก registration กับ sensor_id และ source address จาก recvfrom() ส่วน packet หลังลงทะเบียนต้องมี session_id ที่ server ออกให้ ช่วยแยก logical connection แม้ transport เป็น connectionless

```
virtual sensors -> UDP sockets -> 127.0.0.1:ephemeral
|
v
DART server -> UDP socket -> 127.0.0.1:assigned-port
|
v
worker pool + state
```

SECTION 05

Wire format: fixed header 40 bytes

Network byte order (big-endian), maximum datagram 1,200 bytes, maximum payload 1,160 bytes

Offset	Field	Size	ความหมาย
0	magic	4	ASCII DART
4	version	1	protocol version = 1
5	msg_type	1	ชนิด request/response/data
6	delivery	1	CONTROL/BATCH/LATEST/CRITICAL
7	flags	1	ACK_REQUIRED, RETRANSMISSION, SIMULATED
8	session_id	4	logical session
12	sensor_id	4	ตัวตน sensor
16	sequence	4	identity, ordering, ACK correlation
20	timestamp_ms	8	Unix epoch milliseconds
28	ttl_ms	4	อายุสูงสุดของ message; 0 = no expiry
32	payload_length	2	จำนวน payload bytes
34	status_code	2	0 ใน request; 2xx/4xx/5xx ใน response
36	crc32	4	CRC32 ของ header ที่ zero ช่อง CRC + payload

Syntax / semantics / rules

- Syntax: struct format !4sBBBBIIIIQHII และ payload ตาม msg_type
- Semantics: field ทุกตัวกำหนดความหมายของ session, delivery, ordering, deadline และ result
- Rules: client request ใช้ status 0; server response ใช้ CONTROL; ACK echo sequence; flags ต้องสอดคล้องกับ message

SECTION 06

Message types, request/response และ status

ทุก response แสดง numeric status code พร้อม status phrase ใน log เพื่อให้ตรงโจทย์

Code	Message	ทิศทาง	Response / behavior
1	REGISTER_REQ	C -> S	REGISTER_RES
2	REGISTER_RES	S -> C	201 REGISTERED
3	DATA_BATCH	C -> S	no ACK ใน strict DART
4	LATEST_UPDATE	C -> S	เก็บเฉพาะ sequence ใหม่กว่า
5	CRITICAL_ALERT	C -> S	ACK 202/409/408
6	ACK	S -> C	echo session/sensor/sequence
7/8	CONFIG_REQ/RES	C <-> S	200 OK
9/10	HEARTBEAT/ACK	C <-> S	200 OK
11	ERROR	S -> C	JSON error + 4xx/5xx

Status	Phrase	ใช้เมื่อ
200	OK	config/heartbeat สำเร็จ
201	REGISTERED	สร้างหรือคืน session สำเร็จ
202	ACCEPTED	รับและประมวลผล data สำเร็จ
400	MALFORMED	envelope/header ไม่ตรงกติกา
401	UNREGISTERED	session ไม่ถูกต้อง
408	EXPIRED	เกิน timestamp + TTL
409	DUPLICATE	เคย commit identity นี้แล้ว
422	INVALID PAYLOAD	payload ผิด schema/range
500	INTERNAL ERROR	handler ล้มเหลวโดยไม่เปิดเผย traceback

SECTION 07

Payload และ delivery policy

Binary สำหรับข้อมูล; compact JSON สำหรับ control/alert ที่ต้องอ่านและขยายได้ง่าย

DATA_BATCH

```
count:uint16 + reading[count]  
reading = metric_id:uint8 + value:float32 + age_ms:uint16  
maximum = 165 readings per datagram
```

LATEST_UPDATE

```
metric_id:uint8 + value:float32
```

CRITICAL_ALERT

```
{"alert_type":"FIRE_DETECTED","severity":"critical",  
"value":92.4,"unit":"°C","message":"..."}
```

กฎสำคัญ

- ค่า JSON ต้องเป็น finite number; NaN/Infinity ถูกปฏิเสธ
- Binary float ต้อง represent เป็น finite float32 ได้
- age_ms วัดจาก monotonic capture time ถึงเวลาสร้าง batch และ cap ที่ 65,535 ms
- Strict DART: DATA_BATCH/LATEST_UPDATE ห้ามขอ ACK; CRITICAL/CONTROL ต้องขอ ACK
- Experimental override เปิดเฉพาะ benchmark เพื่อจำลอง raw และ reliable-all โดยไม่ทำให้ strict default อ่อนลง

SECTION 08

Reliability: ACK, timer, retransmission, duplicate

DART ประยุกต์แนวคิด reliable data transfer จาก Chapter 3 เฉพาะข้อความที่คุ้มค่า

```
Client Server
| -- CRITICAL seq=77 -----> | process once
| <--- ACK path (would be 202) [suppressed before sendto] X
| timeout 250 ms |
| -- seq=77 + RETRANSMISSION -> | duplicate cache hit
| <--- ACK 409 DUPLICATE ----- |
| delivery confirmed |
```

Sender rules

- ใช้ sequence เดิมทุก attempt เพื่อให้ ACK correlation และ duplicate identity ตรงกัน
- รอ response ด้วย timer และเพิ่ม timeout แบบ exponential จนถึง max_attempts หรือ TTL
- ยอมรับ ACK เฉพาะ type/status/session/sensor/sequence ที่ผ่าน envelope และ payload validation

Receiver rules

- ตรวจสอบ CRC, version, size, flags, delivery class, status, session และ TTL ก่อนเปลี่ยน state
- Identity = session_id + sensor_id + msg_type + sequence
- Packet ที่กำลัง process อยู่ใช้ condition รอ commit; ถ้า handler แรกล้มเหลว retry สามารถเข้าทำงานแทน

ผลลัพธ์ที่ดึงใจ

เดโมรองรับ ACK ก่อน sendto() เพื่อบังคับ retransmission; duplicate ต้องไม่สร้าง critical side effect ซ้ำภายใน cache window

SECTION 09

โครงสร้าง source code และวิธีรัน

Python standard library เป็นหลัก; ไม่มี database, GUI หรือ hardware dependency

ไฟล์	หน้าที่
dart/protocol.py	packet model, encode/decode, CRC32, payload codecs
dart/client.py	registration, sender, pending response, ACK/retry
dart/server.py	UDP receiver, worker pool, validation, session/state/metrics
dart/network.py	seeded pseudo-random loss/corruption/delay wrapper
dart/simulator.py	virtual sensor schedule, batching, alerts, policy
demo.py	one-command end-to-end proof + PASS oracle
benchmark.py	fixed-workload policy comparison + JSON/CSV
wireshark/dart.lua	DART dissector for UDP port 9999

เริ่มทดลองเร็วที่สุด

```
# run from the repository root
python3 -B demo.py
python3 -B benchmark.py --quick
python3 -B -m unittest discover -s tests -v
```

รันแยก terminal

```
# terminal 1
python3 -B -m dart.server --drop-first-critical-ack
# terminal 2
python3 -B -m dart.simulator --sensors 5 --duration 8
```

SECTION 10

ผลสถิติ end-to-end

หลักฐานจาก 2026-08-28T13:46:34.284597+07:00 - DART policy, 5 virtual sensors, 10% client-side simulated loss

ตัวชี้วัด	ผลจริง	ความหมาย
Sensors registered	5/5	session สำเร็จทุกตัว
Client packets sent	80	datagram ที่ผ่านไปยัง UDP socket
Simulated client drops	19	loss ที่ impairment wrapper สร้าง
Retransmissions	3	sender timeout และส่ง sequence เดิม
Normal readings accepted	125	batch decode และ process สำเร็จ
Critical alerts processed	1	side effect สำคัญเกิดหนึ่งครั้ง
Duplicate packets	1	retry ถูกตรวจว่าเป็น duplicate
ACKs deliberately suppressed	1	ระงับก่อน sendto() เพื่อบังคับ timeout/retry

DEMO RESULT: PASS - เกณฑ์ตรวจ confirmation, process-once ภายใน cache window, forced ACK suppression, retransmission และ duplicate suppression ครบ

Test suite

ผ่าน unit/integration tests 92 กรณี ครอบคลุม wire boundary, batch limit, CRC, JSON/float validation, response envelope, concurrent duplicate, expiry/session, impairment concurrency, simulator lifecycle, CLI boundary, benchmark provenance, demo oracle และ localhost end-to-end

SECTION 11

วิธี benchmark ที่เปรียบเทียบได้

เปรียบเทียบ RAW, RELIABLE-ALL และ DART ภายใต้ logical workload เดียวกัน

เงื่อนไข	ค่า
Transport	UDP over IPv4 loopback
Sensors / duration	5 sensors / 4.0 s ต่อ case
Repeats	5
Loss rates	0%, 5%, 10%, 20%
Seed / cases	base 100; 60 cases; case/simulation/server seeds บันทึกทุก case
Alerts	3 ต่อ sensor; ทุก sensor
Workload equality	precomputed schedule + counts/values/offsets + SHA-256 fingerprint
Bytes	attempted/sent DART bytes สองทิศทาง; รวม setup; ไม่รวม UDP/IP/link header

นิยามที่ต้องแยก

- Server acceptance: server ได้รับและ process critical alert อย่างน้อยครั้งแรก
- Client confirmation: sender ได้ response ที่ยอมรับก่อนหมด attempt/TTL; RAW ไม่มีค่านี้
- Latest delivery: update ทุกอันที่ถึง server เทียบกับ generated
- Latest final state: ค่าสุดท้ายของแต่ละ sensor ถูกต้องหรือไม่ แม้ค่ากลางบางค่าหาย
- Attempted bytes รวม packet ที่ impairment จำลองว่าหาย; sent bytes คือ packet ที่ส่งถึง OS

ข้อควรระวังในการตีความ

Seed กำหนด pseudo-random stream แต่ thread scheduling และจำนวน ACK/retry อาจทำให้ packet ได้รับ random draw ใดไม่เหมือนกัน จึงใช้ repeats และไม่อ้าง paired packet-level trace ผลนี้เป็น loopback experiment ไม่ใช่ Internet-wide performance claim

SECTION 12

ผล benchmark ที่ 20% simulated loss

5 sensors × 4.0 s × 5 repeats; base seed 100; acceptance กับ confirmation เป็นคนละ metric

Policy	Normal	Latest	Final	Crit. accept	Confirmed	B/value	Retry avg
raw	80.0%	77.0%	88.0%	78.7%	n/a	53.9	1.6
reliable-all	100.0%	99.5%	100.0%	100.0%	100.0%	97.2	41.6
dart	76.0%	81.5%	88.0%	100.0%	98.7%	67.3	9.4

สิ่งที่ผลชุดนี้สนับสนุน

- RAW ใช้ 53.9 B/value แต่ไม่มี client confirmation และ critical acceptance 78.7% (59 จาก 75 events)
- RELIABLE-ALL ยืนยัน critical 100.0% และ latest delivery 99.5% โดยใช้ 97.2 B/value กับ retry เฉลี่ย 41.6
- DART ยืนยัน critical 98.7% (74 จาก 75 events) และใช้ 67.3 B/value โดยยอมให้ normal/latest บางส่วนสูญหาย
- Latest final state เป็นเป้าหมายที่ต่างจาก delivery ทุก update; loss ของ update กลางอาจไม่ทำให้ state สุดท้ายผิด
- ที่ 20% loss มี server-accept samples: raw 59, reliable-all 75, DART 75; ACK samples ของ DART 74

Latency

Policy	Critical server accept p95	Critical ACK p95
raw	1.0 ms	n/a
reliable-all	256.0 ms	759.2 ms
dart	255.3 ms	757.2 ms

จุดเด่นของ DART คือ trade-off ที่อธิบายและวัดได้ ไม่ใช่ชนะทุก metric

SECTION 13

Wireshark และสิ่งที่นำเสนอในคลิป

DART dissector ทำให้เห็น field, status และ payload บน wire แทนการอธิบายจาก code อย่างเดียว

Wireshark filters

```
dart
dart.msg_type == 5
dart.flags.ack_required == 1
dart.flags.retransmission == 1
dart.status_code == 409
```

ลำดับภาพที่ควร capture

- REGISTER_REQ -> REGISTER_RES 201 เพื่อแสดง session
- DATA_BATCH เพื่อเปิดดู count และ binary readings
- LATEST_UPDATE หลาย sequence เพื่ออธิบาย newest-state semantics
- CRITICAL_ALERT seq เดิมสองครั้ง แล้ว ACK 409 DUPLICATE เพื่อพิสูจน์ retry + process-once ภายใน duplicate-cache window

แผนคลิปไม่เกิน 15 นาที

เวลา	เนื้อหา
0:00-2:05	ปัญหา แนวคิด และขอบเขตโปรเจกต์
2:05-5:25	UDP, delivery classes, header และ message flow
5:25-7:05	ACK/retry/duplicate และ concurrency
7:05-8:00	code walkthrough เฉพาะส่วน protocol
8:00-11:10	live demo, test variations และ Wireshark
11:10-14:15	benchmark, จุดเด่น, limitations และสรุป

หมายเหตุ: ผู้ส่งต้องอัดวิดีโอและปรากฏตัวระหว่างบรรยายบางส่วนตามใบโจทย์ ระบบนี้เตรียมสไลด์ สคริปต์ และ demo ให้ แต่ไม่สามารถแทนการปรากฏตัวของนิสิตในวิดีโอได้

SECTION 14

ความเชื่อมโยง Chapter 1-3

การออกแบบทุกส่วนผูกกับแนวคิดในเนื้อหาวิชา ไม่ใช่เพิ่ม feature แอปที่ไม่เกี่ยวกับ network

Chapter	แนวคิด	หลักฐานใน DART
1	protocol = format + order + actions	40-byte header, message flow, timeout/retry rules
1	delay, loss, throughput	loss/delay wrapper, latency, delivery และ bytes/value metrics
1	layering/encapsulation	DART message -> UDP datagram -> IP packet
2	client-server, process, socket	SensorClient/DartServer และ IP+port
2	transport service requirements	เหตุผลเลือก UDP และแบ่ง delivery class
2	application protocol syntax/semantics	wire format, status, payload schema, rules
3	UDP mux/demux/checksum	หลาย client address, UDP port, CRC32 เพิ่มระดับ application
3	reliable data transfer	sequence, ACK, timer, retransmission, duplicate suppression

การวางตำแหน่งงาน

DART ไม่ใช่ dashboard และคะแนนไม่ควรพึ่งความสวยของ app โปรแกรม simulator มีหน้าที่เป็น client ที่สร้าง traffic เพื่อพิสูจน์ protocol ส่วน demo output และ Wireshark แสดงว่า packet ทำงานตามกติกา จุดสร้างสรรค์อยู่ที่แนวคิด deadline-aware selective reliability และวิธีนำเสนอหลักฐาน

SECTION 15

จุดเด่น ข้อจำกัด และบทสรุป

โปรโตคอลที่ได้อธิบายได้ทั้งสิ่งที่ทำได้ สิ่งที่แลก และสิ่งที่ยังไม่ทำ

จุดเด่น

- Reliability เป็นการตัดสินใจระดับ message: batch, latest และ critical ไม่ถูกปฏิบัติเหมือนกัน
- Wire format ตรวจสอบได้จริง มี Wireshark dissector และ status code/status phrase
- รองรับหลาย virtual sensors พร้อมกันและจัดการ concurrent duplicate อย่างปลอดภัย
- Benchmark แยก acceptance/confirmation/final state/bytes ทำให้ claim ตรวจสอบได้

เทียบเชิงแนวคิดกับวิธีที่มีอยู่

Protocol	Semantics หลัก	DART ต่างอย่างไร
TCP	reliable ordered byte stream	คง datagram boundary และเลือก reliability ต่อ message
MQTT	broker + QoS 0/1/2	prototype แบบ client-server ตรง ไม่ใช่ broker
CoAP	Confirmable / Non-confirmable	เพิ่ม batch/latest/critical policy กับ TTL

ข้อจำกัด

- ยังไม่มี authentication, encryption, replay protection ระยะยาว หรือ congestion control
- Session อยู่ใน memory และเหมาะกับ prototype/ห้องเรียน ไม่ใช่ production deployment
- Loopback impairment เป็นแบบจำลองผล loss ไม่ใช่ topology/router queue จริง
- Reliable call เป็น stop-and-wait; workload ที่ throughput สูงควรพัฒนา windowing/selective ACK

**DART ทำให้ reliability เป็นตัวเลือกของข้อความ: ปกติให้เบา ล่าสุดให้สด
เหตุสำคัญให้ยืนยัน**

ก่อนส่ง

- ระหว่างอัด รัน demo.py และถ่ายหน้าจอ terminal + Wireshark
- อัปเดตคลิปไม่เกิน 15 นาที และให้เห็นผู้นำเสนอบางส่วน
- อัปโหลดและตรวจว่า link PDF, source code และ video เปิดได้จากบัญชีอื่น

แหล่งอ้างอิงภายในงาน

- 1) Project 1_ Socket Programming.pdf - ใบโงทย์ที่ผู้จัดทำได้รับ (ไม่รวมใน public repository)
- 2) เอกสารการเรียนรู้ Chapter 1-3 ที่ผู้จัดทำได้รับ (ไม่รวมใน public repository)
- 3) Source code, protocol specification และผลทดลองอยู่ที่ repository root
- 4) RFC 9293 (TCP), OASIS MQTT 5.0 และ RFC 7252 (CoAP) สำหรับตารางเปรียบเทียบ